Solution: Maximum Swap

Problem Statement

Given a non-negative integer `num`, return the `maximum` number, which you can create by `swapping` any two digits of the number only once. If no swaps can improve the number, return the original number.

Examples

1. **Example 1**:
    - **Input**: `2736`
    - **Expected Output**: `7236`
    - **Justification**: Swapping the first and second digits (`2` and `7`) results in the largest possible number.
2. **Example 2**:
    - **Input**: `9965`
    - **Expected Output**: `9965`
    - **Justification**: The number is already in its maximum form, so no swap is needed.
3. **Example 3**:
    - **Input**: `7281912`
    - **Expected Output**: `9281712`
    - **Justification**: Swapping the first digit (`7`) with the first `9` found from the left results in the maximum number.

Solution

To solve this problem, the approach revolves around identifying the most significant digit that can be increased by a swap. We traverse the number from right to left, identifying the largest digit seen so far and its index. This is because a larger digit appearing later in the number, when swapped with a smaller digit to its left, will yield a greater value. The key is to find the leftmost digit for which a larger digit exists anywhere to its right. Swapping these two digits will result in the maximum possible number. This approach is effective as it minimizes the swaps and ensures the largest digit possible in the highest place value.

Step-by-Step Algorithm

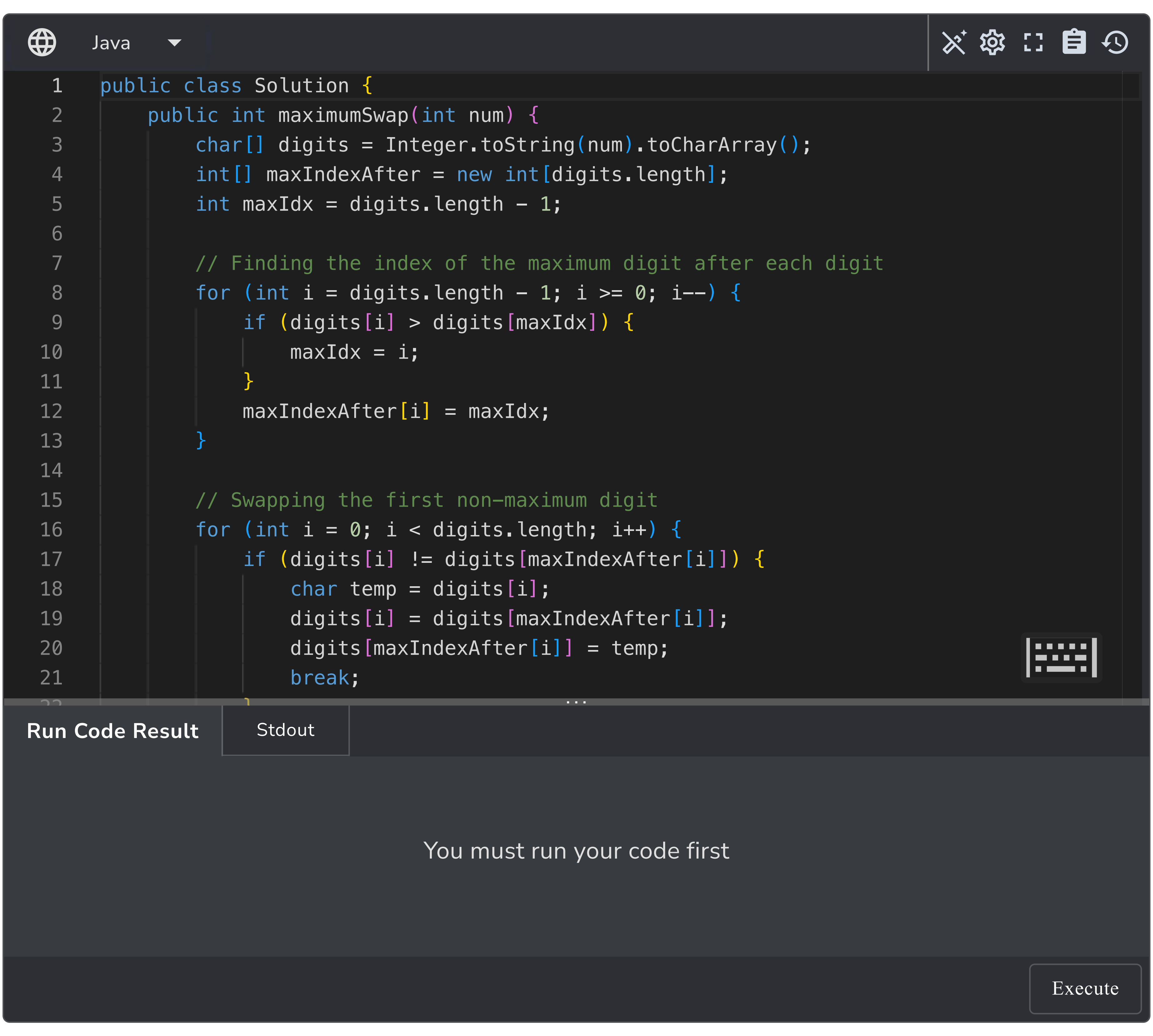
1. Convert the given number into an array of its digits for easier manipulation.
2. **Prepare the `maxIndexAfter` Array**:
    - Initialize an array `maxIndexAfter` to record the index of the maximum digit found to the right of each digit (including itself).
    - Traverse the digit array from right to left.
    - Update `maxIndexAfter` for each position with the index of the largest digit seen so far.
3. **Perform the Swap**:
    - Traverse the `digits` array from left to right.
    - If `digits[i]` and `digits[maxIndexAfter[i]]` are not same, swap both elements.
    - After the first swap, break the loop.
4. **Reconstruct the Number**: Convert the digit array back into a single integer.
5. **Return the Result**: Provide the modified number as the output.

Algorithm Walkthrough

Consider the Input `7281912`.

1. **Initialization**:
    - Convert to digit array: `[7, 2, 8, 1, 9, 1, 2]`.
    - Initialize `maxIndexAfter` array with the same length as the digit array.
2. **Preparing the `maxIndexAfter` Array**:
    - Start from the rightmost digit.
    - Iteration 1 (Index 6): Digit `2`, `maxIndexAfter[6] = 6`.
    - Iteration 2 (Index 5): Digit `1`, `maxIndexAfter[5] = 6` (as `2` is the maximum seen so far).
    - Iteration 3 (Index 4): Digit `9`, `maxIndexAfter[4] = 4`.
    - Continue updating `maxIndexAfter` for each digit.
    - Final `maxIndexAfter` array: `[4, 4, 4, 4, 4, 6, 6]`.
3. **Performing the Swap**:
    - Traverse from left to right.
    - Iteration 1 (Index 0): Digit `7`, compare with `maxIndexAfter[0]` (digit at index 4 which is `9`), so swap them.
    - As soon as the first swap opportunity is found, break the loop.
    - Array after swap: `[9, 2, 8, 1, 7, 1, 2]`.
4. **Reconstructing the Number**:
    - Convert array back to number: `9281712`.
5. **Returning the Result**:
    - Return `9281712`.

Code



```java
public class Solution {
    public int maximumSwap(int num) {
        char[] digits = Integer.toString(num).toCharArray();
        int[] maxIndexAfter = new int[digits.length];
        int maxIdx = digits.length - 1;

        // Finding the index of the maximum digit after each digit
        for (int i = digits.length - 1; i >= 0; i--) {
            if (digits[i] > digits[maxIdx]) {
                maxIdx = i;
            }
            maxIndexAfter[i] = maxIdx;
        }

        // Swapping the first non-maximum digit
        for (int i = 0; i < digits.length; i++) {
            if (digits[i] != digits[maxIndexAfter[i]]) {
                char temp = digits[i];
                digits[i] = digits[maxIndexAfter[i]];
                digits[maxIndexAfter[i]] = temp;
                break;
```

You must run your code first

Complexity Analysis

Time Complexity: O(n)

The process of converting the number to a digit array and traversing it (both for preparing `maxIndexAfter` and finding the swap position) each take `O(n)` time, linearly dependent on the number of digits (`n`).

Space Complexity: O(n)

The algorithm requires space proportional to the number of digits (`n`) for storing the digit array and the `maxIndexAfter` array, leading to `O(n)` space complexity.

← Previous
Maximum Swap (Medium)
Next →
Custom Sort String (Medium)
☐ Mark as Completed